university of groningen

Models and Semantics of Computation (MaSC)

Lecture 12: The π-calculus

Jorge A. Pérez

Bernoulli Institute for Mathematics, Computer Science, and AI
University of Groningen, Groningen, the Netherlands

October 19, 2026

Up To Here

Sequential computation (IMP/HOFL)

- determinacy
- any two non-terminating programs are equivalent

Up To Here



Sequential computation (IMP/HOFL)

- determinacy
- any two non-terminating programs are equivalent

Concurrent computation (CCS)

- interaction is a primary issue
- exhibit intrinsic nondeterminism to external observers
- non-termination can be a desirable feature (e.g. servers)
- not all non-terminating processes are equivalent
  $\rightarrow$ new notions of behavioral equivalence needed

Up To Here



Sequential computation (IMP/HOFL)

- determinacy
- any two non-terminating programs are equivalent

Concurrent computation (CCS)

- interaction is a primary issue
- exhibit intrinsic nondeterminism to external observers
- non-termination can be a desirable feature (e.g. servers)
- not all non-terminating processes are equivalent
  $\rightarrow$ new notions of behavioral equivalence needed

Today: Concurrency in the π-calculus

- Communication with name-passing
- Name mobility: Dynamic communication topologies (scope extrusion)
- Syntax and operational semantics



Name Mobility

Name Mobility



- In CCS, communication is statically defined: channels cannot be exchanged.
- What if channels are treated as ordinary values?
- How to extend CCS to exchange communication means?

Name Mobility



- In CCS, communication is statically defined: channels cannot be exchanged.
- What if channels are treated as ordinary values?
- How to extend CCS to exchange communication means?

The π-calculus (Milner, Parrow, Walker; 1992)

Name Mobility



- In CCS, communication is statically defined: channels cannot be exchanged.
- What if channels are treated as ordinary values?
- How to extend CCS to exchange communication means?

The π-calculus (Milner, Parrow, Walker; 1992)

- As in CCS, communication channels can exchange values.
- **Insight**: A single class of objects for channels and values: a set of names, ranged over $x, y, z, \ldots$. Processes exchange names and can create new ones.

Name Mobility



- In CCS, communication is statically defined: channels cannot be exchanged.
- What if channels are treated as ordinary values?
- How to extend CCS to exchange communication means?

The π-calculus (Milner, Parrow, Walker; 1992)

- As in CCS, communication channels can exchange values.
- **Insight**: A single class of objects for channels and values: a set of names, ranged over $x, y, z, \ldots$. Processes exchange names and can create new ones.
- **Name mobility:** Channels/names can be exchanged in interactions.

Name Mobility



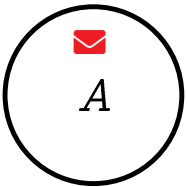
- In CCS, communication is statically defined: channels cannot be exchanged.
- What if channels are treated as ordinary values?
- How to extend CCS to exchange communication means?

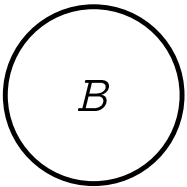
The π-calculus (Milner, Parrow, Walker; 1992)

- As in CCS, communication channels can exchange values.
- **Insight**: A single class of objects for channels and values: a set of names, ranged over $x, y, z, \ldots$. Processes exchange names and can create new ones.
- **Name mobility:** Channels/names can be exchanged in interactions.
- Very influential: widely regarded as the counterpart of the λ-calculus for concurrency and interaction.

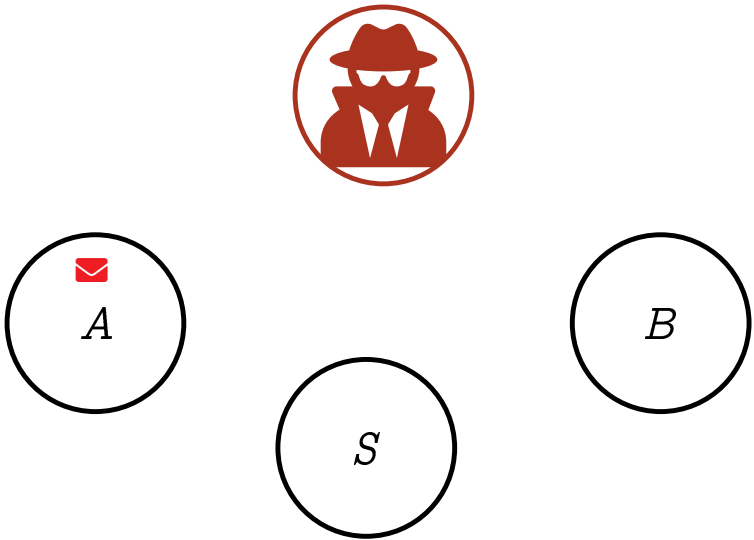
Name Mobility



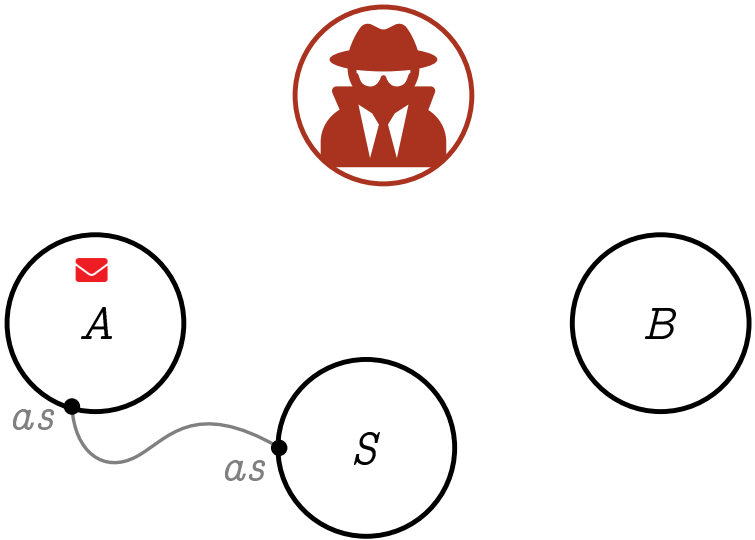




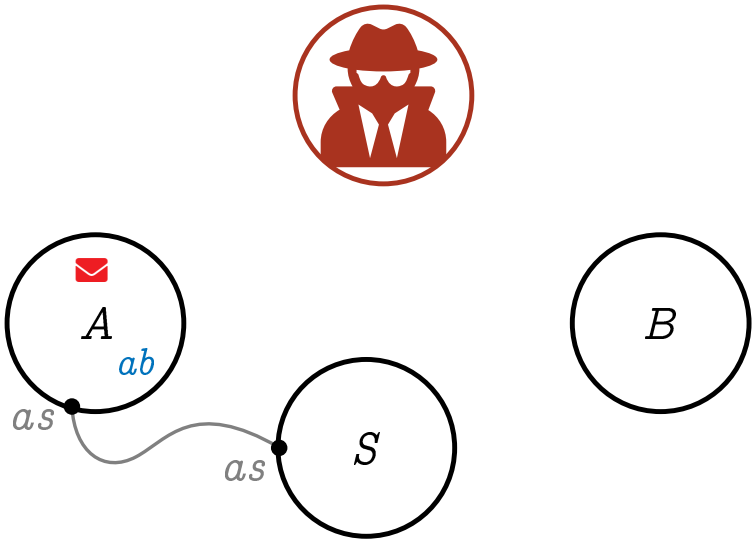
Name Mobility



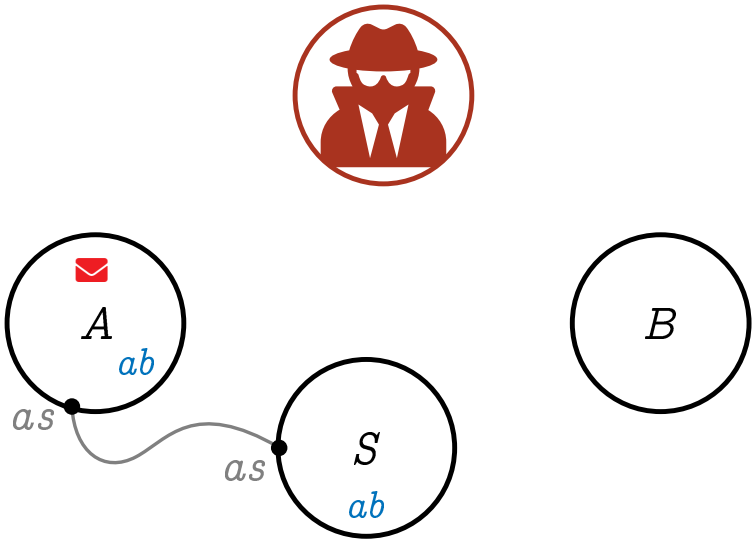
Name Mobility



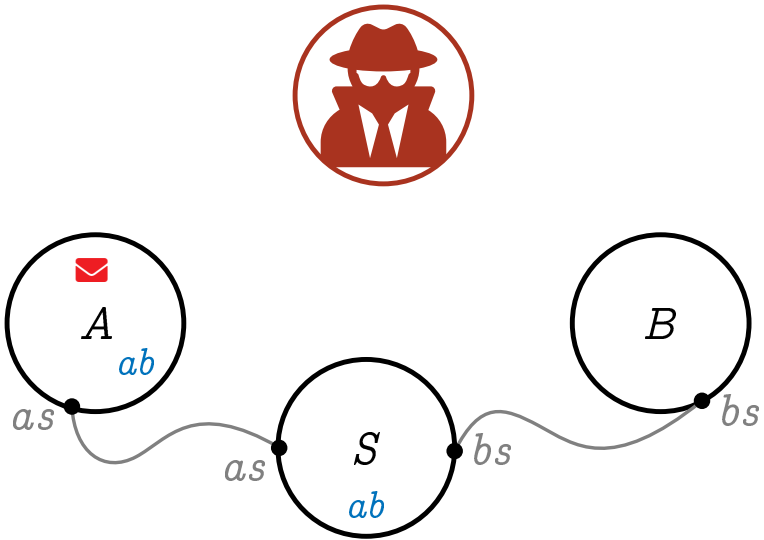
Name Mobility



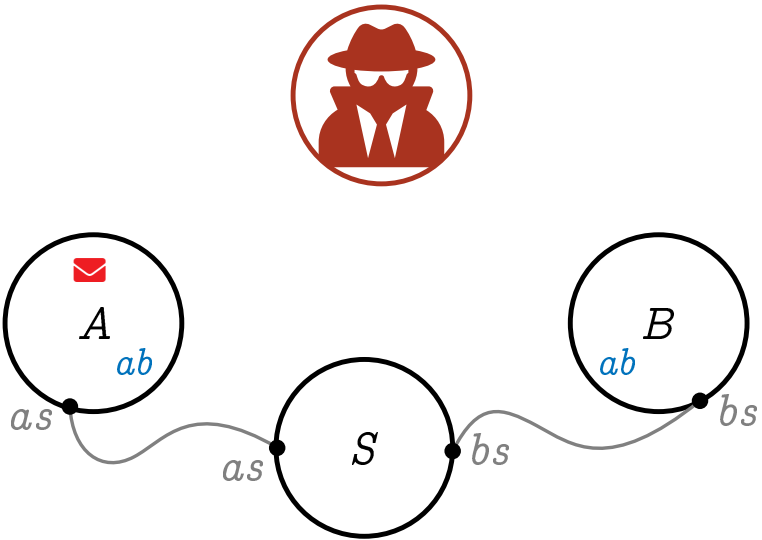
Name Mobility



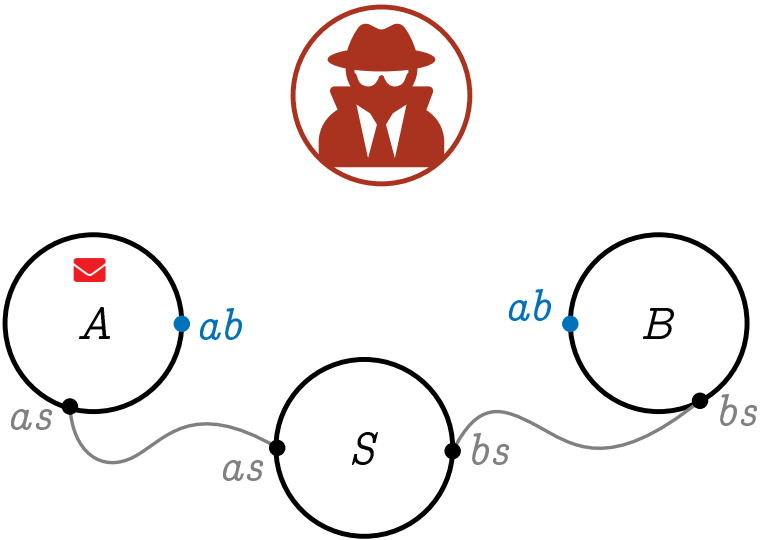
Name Mobility



Name Mobility

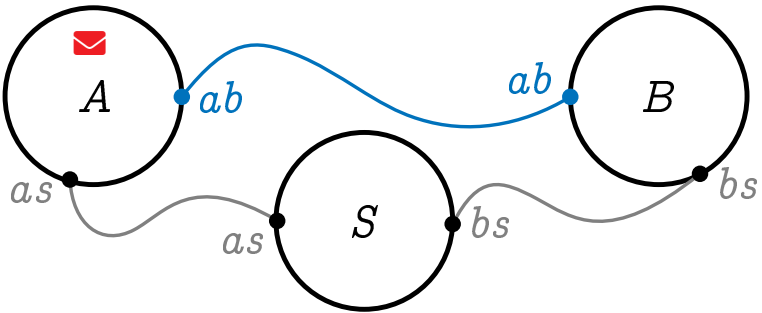


Name Mobility



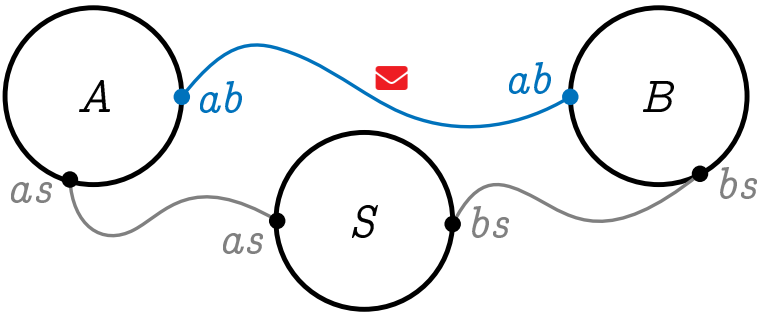
Name Mobility





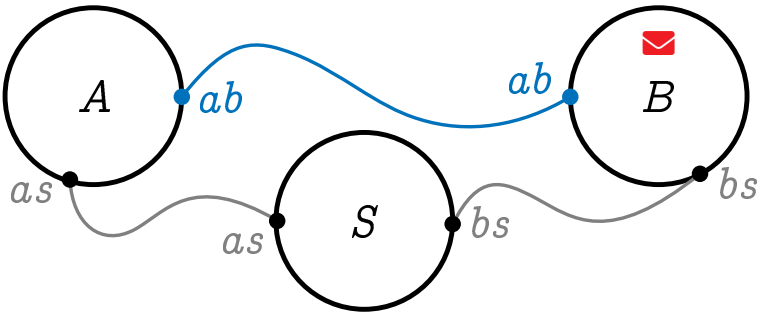
Name Mobility





Name Mobility







The π-calculus

The π-calculus: Syntax



π	$::= \overline{x}\, y$	send y on x
	$\mid x(y)$	receive on x and store in y
	$\mid \tau$	internal action
p, q	$::= \mathrm{nil}$	inactive process
	$\mid \pi.p$	action prefix
	$\mid [x = y]p$	matching (think: if $x = y$ then p)
	$\mid p + q$	non deterministic choice
	$\mid p \mid q$	parallel composition
	$\mid (y)p$	restriction (similar to $p \backslash y$, name y is local to p)
	$\mid !p$	replication (think: $p \mid p \mid \cdots$)

The π-calculus: Syntax



Prefixes	$\pi ::= \overline{x}\, y \mid x(y) \mid \tau$
Processes	$p, q ::= \mathrm{nil} \mid \pi.p \mid [x = y]p \mid p + q \mid p \mid q \mid (y)p \mid\, !p$

- The only binders are input prefix and name restriction:

$$x(y).p \qquad (y)p$$

- The sets of free and bound names of p are denoted $\mathrm{fn}(p)$ and $\mathrm{bn}(p)$.
- Capture-avoiding substitution is denoted $p[{}^{y}/_{x}]$.
- Processes are taken up to alpha-renaming of bound names:

$$x(y).p = x(z).p[{}^{z}/_{y}] \quad \text{if } z \notin \mathrm{fn}(p)$$
$$(y)p = (z)p[{}^{z}/_{y}] \quad \text{if } z \notin \mathrm{fn}(p)$$

The π-calculus: Useful Notations



$\sum_{i=1}^{n} p_i \triangleq p_1 + \cdots + p_n$	generalized sums
$\prod_{i=1}^{n} p_i \triangleq p_1 \mid \cdots \mid p_n$	generalized parallel compositions
$(\tilde{a})p \triangleq (a_1)\cdots(a_n)p$	generalized name restriction
$!x(y).p$	input-guarded replication

The π-calculus: Conventions



- We omit trailing $\mathbf{nil}$. Example:

$$(y)\overline{x}\,y.\overline{y}\,m.\mathbf{nil} \quad \text{as} \quad (y)\overline{x}\,y.\overline{y}\,m$$

The π-calculus: Conventions



- We omit trailing $\mathbf{nil}$. Example:

$$(y)\overline{x}\,y.\overline{y}\,m.\mathbf{nil} \quad \text{as} \quad (y)\overline{x}\,y.\overline{y}\,m$$

- Actions with vacuous messages are written as CCS processes. Example:

$$\overline{x} \mid x.p + x.q$$

The π-calculus: Conventions



- We omit trailing $\mathbf{nil}$. Example:

$$(y)\overline{x}\,y.\overline{y}\,m.\mathbf{nil} \quad \text{as} \quad (y)\overline{x}\,y.\overline{y}\,m$$

- Actions with vacuous messages are written as CCS processes. Example:

$$\overline{x} \mid x.p + x.q$$

- Name mismatching can be represented (provided finite cases):

$$[x \neq y_i]p \quad \text{as} \quad \sum_{j \neq i} [x = y_j]p$$

The π-calculus: Conventions



- We omit trailing $\mathbf{nil}$. Example:

$$(y)\overline{x}\,y.\overline{y}\,m.\mathbf{nil} \quad \text{as} \quad (y)\overline{x}\,y.\overline{y}\,m$$

- Actions with vacuous messages are written as CCS processes. Example:

$$\overline{x} \mid x.p + x.q$$

- Name mismatching can be represented (provided finite cases):

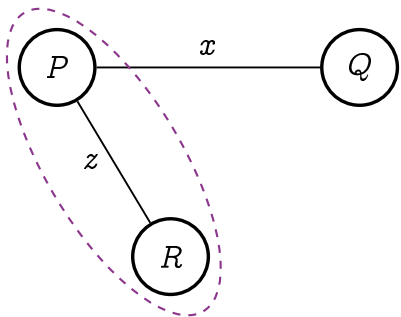
$$[x \neq y_i]p \quad \text{as} \quad \sum_{j \neq i} [x = y_j]p$$

- Polyadic communication, i.e., exchanging lists of names (can be encoded):

$$\overline{x}\langle \underbrace{z_1, \ldots, z_n}_{\tilde{z}} \rangle.p \qquad x(\underbrace{y_1, \ldots, y_n}_{\tilde{y}}).q$$

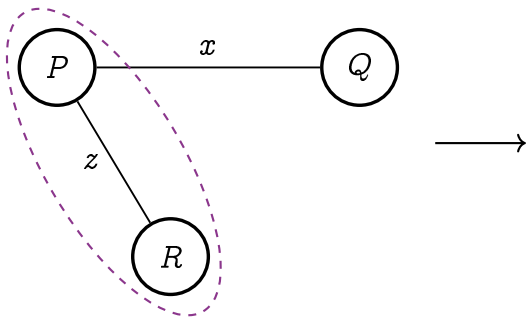
Scope Extrusion

- A process $(z)(P \mid R) \mid Q$, with x shared between P and Q.
  Intuition: z is a key private to P, which grants access to resource R.



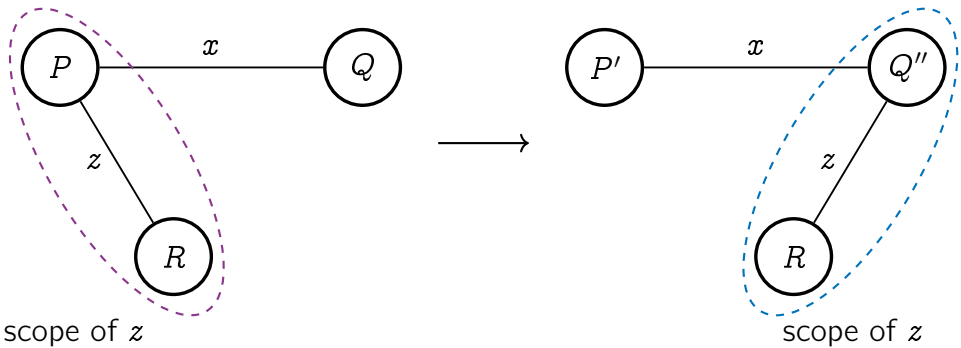
Scope Extrusion

- A process $(z)(P \mid R) \mid Q$, with x shared between P and Q.
  Intuition: z is a key private to P, which grants access to resource R.
- If P sends z over x we obtain $P' \mid (z)(R \mid Q'')$ and so the scope of z evolves.
  Access to R has moved from P to Q.



Scope Extrusion

- A process $(z)(P \mid R) \mid Q$, with x shared between P and Q.
  Intuition: z is a key private to P, which grants access to resource R.
- If P sends z over x we obtain $P' \mid (z)(R \mid Q'')$ and so the scope of z evolves.
  Access to R has moved from P to Q.



Example

$$N \triangleq (as)(bs)(A \mid S \mid B)$$

$$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$$

$$S \triangleq\ !\ as(x).\overline{bs}\ x$$

$$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$$

Example

$$N \triangleq (as)(bs)(A \mid S \mid B)$$

$$A \triangleq (ab)\,\overline{as}\,ab.\,\overline{ab}\,m.\,A'$$

create a new (private) channel ab

$$S \triangleq !\,as(x).\overline{bs}\,x$$

$$B \triangleq bs(y).\,y(w).\,B'$$

Example

$$N \triangleq (as)(bs)(A \mid S \mid B)$$

$$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$$

send ab on as

create a new (private) channel ab

$$S \triangleq\ !\ as(x).\overline{bs}\ x$$

$$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$$

Example

$$N \triangleq (as)(bs)(A \mid S \mid B)$$

$$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$$

send m on ab

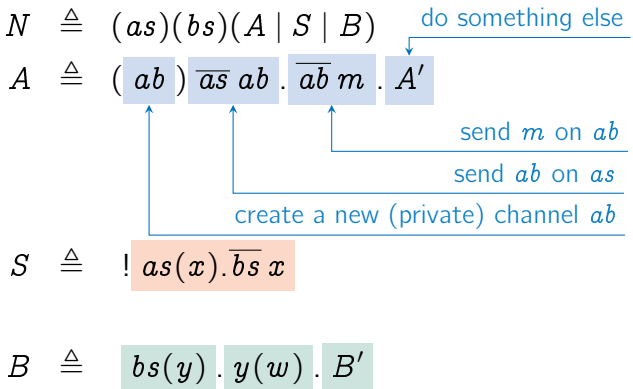
send ab on as

create a new (private) channel ab

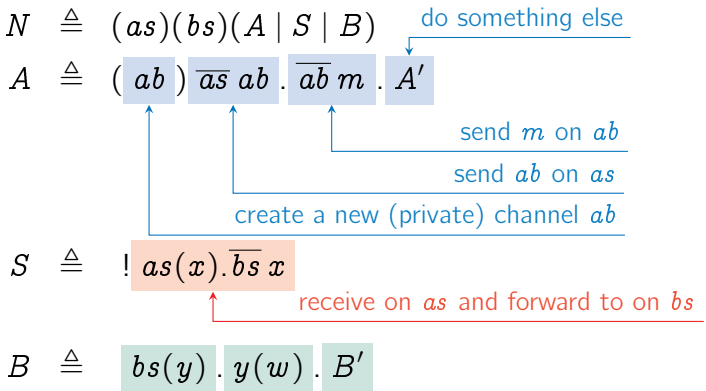
$$S \triangleq\ !\ as(x).\overline{bs}\ x$$

$$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$$

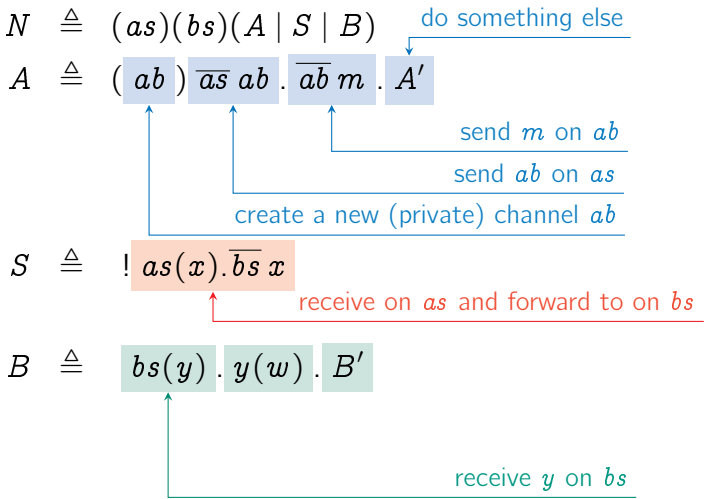
Example


$N \triangleq (as)(bs)(A \mid S \mid B)$
do something else
$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$
send m on ab
send ab on as
create a new (private) channel ab
$S \triangleq\ !\ as(x).\overline{bs}\ x$
$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$

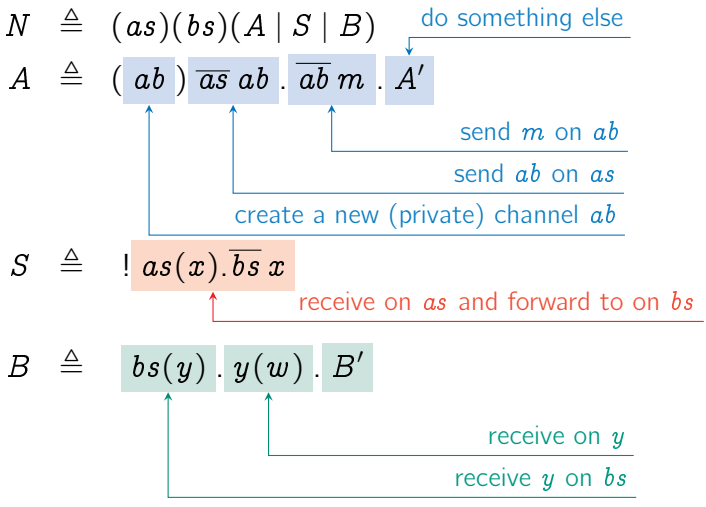
Example


$N \triangleq (as)(bs)(A \mid S \mid B)$
do something else
$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$
send m on ab
send ab on as
create a new (private) channel ab
$S \triangleq\ !\ as(x).\overline{bs}\ x$
receive on as and forward to on bs
$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$

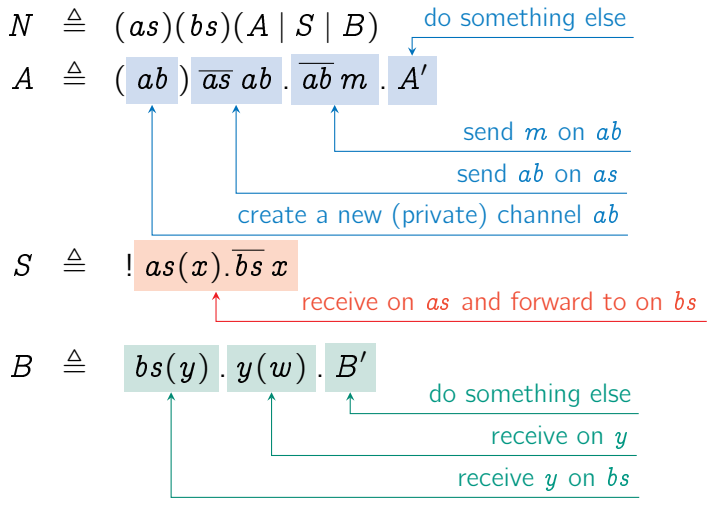
Example


$N \triangleq (as)(bs)(A \mid S \mid B)$
do something else
$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$
send m on ab
send ab on as
create a new (private) channel ab
$S \triangleq\ !\ as(x).\overline{bs}\ x$
receive on as and forward to on bs
$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$
receive y on bs

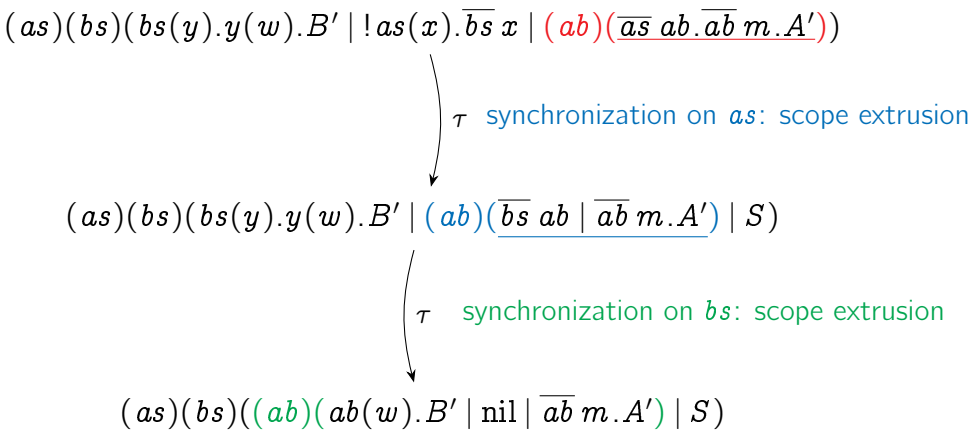
Example


$N \triangleq (as)(bs)(A \mid S \mid B)$
do something else
$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$
send m on ab
send ab on as
create a new (private) channel ab
$S \triangleq\ !\ as(x).\overline{bs}\ x$
receive on as and forward to on bs
$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$
receive on y
receive y on bs

Example


$N \triangleq (as)(bs)(A \mid S \mid B)$
do something else
$A \triangleq (\ ab\ )\ \overline{as}\ ab\ .\ \overline{ab}\ m\ .\ A'$
send m on ab
send ab on as
create a new (private) channel ab
$S \triangleq\ !\ as(x).\overline{bs}\ x$
receive on as and forward to on bs
$B \triangleq\ bs(y)\ .\ y(w)\ .\ B'$
do something else
receive on y
receive y on bs

Example: Dynamic Topologies



Exercise



Three agents: a **printing server**, a **client**, and a **printer**.
The client wishes to print a document d.

- The client and the server share a public name b.
- The server and the printer share a name a.
- The client doesn't share names with the printer, so it cannot contact it.
- The document d cannot be transmitted along public names.

Exercise

One possible specification.

The client, the server, and the printer in three different processes:

$$C \triangleq (r)\overline{b}\, r.r(y).(s)(\overline{y}\, s.s(c).\overline{s}\, d)$$

$$S \triangleq b(x).\overline{x}\, a$$

$$P \triangleq a(u).\overline{u}\, c.u(w).\overline{print}\, w$$

$$N \triangleq C \mid S \mid P$$

Exercise

One possible specification.

The client, the server, and the printer in three different processes:

$$C \triangleq (r)\overline{b}\, r.r(y).(s)(\overline{y}\, s.s(c).\overline{s}\, d)$$
$$S \triangleq b(x).\overline{x}\, a$$
$$P \triangleq a(u).\overline{u}\, c.u(w).\overline{print}\, w$$
$$N \triangleq C \mid S \mid P$$

- C uses private names (r, s) to send requests: first to S and then to P.

Exercise

One possible specification.

The client, the server, and the printer in three different processes:

$$C \triangleq (r)\overline{b}\, r.r(y).(s)(\overline{y}\, s.s(c).\overline{s}\, d)$$
$$S \triangleq b(x).\overline{x}\, a$$
$$P \triangleq a(u).\overline{u}\, c.u(w).\overline{print}\, w$$
$$N \triangleq C \mid S \mid P$$

- C uses private names (r, s) to send requests: first to S and then to P.
- For simplicity, S ends after receiving C's request.
  By design of C, name a is exchanged on a private name (r).

Exercise

One possible specification.

The client, the server, and the printer in three different processes:

$$C \triangleq (r)\overline{b}\, r.r(y).(s)(\overline{y}\, s.s(c).\overline{s}\, d)$$
$$S \triangleq b(x).\overline{x}\, a$$
$$P \triangleq a(u).\overline{u}\, c.u(w).\overline{print}\, w$$
$$N \triangleq C \mid S \mid P$$

- C uses private names (r, s) to send requests: first to S and then to P.
- For simplicity, S ends after receiving C's request.
  By design of C, name a is exchanged on a private name (r).
- P sends a confirmation c after receiving a request on a.
  All exchanges are along s, private to C and P.

Exercise

One possible specification.

The client, the server, and the printer in three different processes:

$$C \triangleq (r)\overline{b}\, r.r(y).(s)(\overline{y}\, s.s(c).\overline{s}\, d)$$
$$S \triangleq b(x).\overline{x}\, a$$
$$P \triangleq a(u).\overline{u}\, c.u(w).\overline{print}\, w$$
$$N \triangleq C \mid S \mid P$$

- C uses private names (r, s) to send requests: first to S and then to P.
- For simplicity, S ends after receiving C's request.
  By design of C, name a is exchanged on a private name (r).
- P sends a confirmation c after receiving a request on a.
  All exchanges are along s, private to C and P.
- The entire network N is the composition of the three processes.

A Summation Service



A client provides a number n and a name s, and the server returns $n + 2$ over s.
We use polyadic communication and assume an auxiliary service $plusOne$:

$$!plusTwo(n, s).(r_1, r_2)(\overline{plusOne}\langle n, r_1\rangle \mid r_1(y).\overline{plusOne}\langle y, r_2\rangle \mid r_2(z).\overline{s}\, z)$$

The service creates private names r_1, r_2 for two sequential calls to $plusOne$.
What does a client process look like?

A Summation Service



A client provides a number n and a name s, and the server returns $n + 2$ over s.
We use polyadic communication and assume an auxiliary service $plusOne$:

$$!plusTwo(n, s).(r_1, r_2)(\overline{plusOne}\langle n, r_1\rangle \mid r_1(y).\overline{plusOne}\langle y, r_2\rangle \mid r_2(z).\overline{s}\, z)$$

The service creates private names r_1, r_2 for two sequential calls to $plusOne$.
What does a client process look like?

An **optimized service**:

$$!plusTwo(n, s).(r)(\overline{plusOne}\langle n, r\rangle \mid r(y).\overline{plusOne}\langle y, s\rangle)$$

Where is the difference?



Operational Semantics

Two Approaches

Reduction Semantics

$$p \longmapsto q$$

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.

Two Approaches



Reduction Semantics

$$p \longmapsto q$$

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.
- Small-step style, more complex than in CCS, due to name passing and scope extrusion.
- Different choices possible, to treat when substitutions occur.

Two Approaches



Reduction Semantics

$$p \longmapsto q$$

- A computation step done without the environment.

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.
- Small-step style, more complex than in CCS, due to name passing and scope extrusion.
- Different choices possible, to treat when substitutions occur.

Two Approaches



Reduction Semantics

$$p \longmapsto q$$

- A computation step done without the environment.
- Big-step style: p includes all components needed to interact.

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.
- Small-step style, more complex than in CCS, due to name passing and scope extrusion.
- Different choices possible, to treat when substitutions occur.

Two Approaches



Reduction Semantics

$$p \longmapsto q$$

- A computation step done without the environment.
- Big-step style: p includes all components needed to interact.
- Uses structural congruence, which rewrites processes to promote interaction.

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.
- Small-step style, more complex than in CCS, due to name passing and scope extrusion.
- Different choices possible, to treat when substitutions occur.

Two Approaches

Reduction Semantics

$$p \longmapsto q$$

- A computation step done without the environment.
- Big-step style: p includes all components needed to interact.
- Uses structural congruence, which rewrites processes to promote interaction.

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

- The label α describes interaction with the environment.
- Small-step style, more complex than in CCS, due to name passing and scope extrusion.
- Different choices possible, to treat when substitutions occur.

The approaches are related:
A reduction $p \longmapsto p'$ corresponds to a transition $p \xrightarrow{\tau} p'$ (and vice versa).

Structural Congruence



The least congruence over processes that includes α-conversion of bound names and that is induced by the following axioms:

$$p + \mathrm{nil} \equiv p \qquad p + q \equiv q + p \qquad (p + q) + r \equiv p + (q + r)$$

$$p \mid \mathrm{nil} \equiv p \qquad p \mid q \equiv q \mid p \qquad (p \mid q) \mid r \equiv p \mid (q \mid r)$$

$$(x)\mathrm{nil} \equiv \mathrm{nil} \qquad (x)(y)p \equiv (y)(x)p \qquad (x)(p \mid q) \equiv p \mid (x)q \quad \text{if } x \notin \mathsf{fn}(p)$$

$$[x = y]\mathrm{nil} \equiv \mathrm{nil} \qquad [x = x]p \equiv p \qquad !p \equiv p \mid !p$$

Structural Congruence

The least congruence over processes that includes α-conversion of bound names and that is induced by the following axioms:

$$
\begin{array}{ccc}
p + \mathrm{nil} \equiv p & p + q \equiv q + p & (p + q) + r \equiv p + (q + r) \\
p \mid \mathrm{nil} \equiv p & p \mid q \equiv q \mid p & (p \mid q) \mid r \equiv p \mid (q \mid r) \\
(x)\mathrm{nil} \equiv \mathrm{nil} & (x)(y)p \equiv (y)(x)p & (x)(p \mid q) \equiv p \mid (x)q \quad \text{if } x \notin \mathrm{fn}(p) \\
[x = y]\mathrm{nil} \equiv \mathrm{nil} & [x = x]p \equiv p & !p \equiv p \mid !p
\end{array}
$$

Intuitively

$P \equiv Q$ holds if Q can be obtained by applying the axioms any number of times, in both directions, to arbitrary sub-processes of P.

Two Remarkable Axioms

Scope Extrusion / Intrusion

$$(x)(p \mid q) \equiv p \mid (x)q \qquad \text{if } x \notin \mathrm{fn}(p)$$

- From left to right, the scope of x shrinks: This is safe, since $x \notin \mathrm{fn}(p)$.
- From right to left, the scope of x enlarges: This is safe (and innocuous), since $x \notin \mathrm{fn}(p)$, the behavior of p is not affected with the enlarged scope.

Two Remarkable Axioms



Scope Extrusion / Intrusion

$$(x)(p \mid q) \equiv p \mid (x)q \qquad \text{if } x \notin \mathrm{fn}(p)$$

- From left to right, the scope of x shrinks: This is safe, since $x \notin \mathrm{fn}(p)$.
- From right to left, the scope of x enlarges: This is safe (and innocuous), since $x \notin \mathrm{fn}(p)$, the behavior of p is not affected with the enlarged scope.

Replication

$$!p \equiv p \mid !p$$

- From left to right, this axiom allows us to "extract" one extra copy of p.
- Inspired by the bang modality in Girard's linear logic (denoted $!A$)

Reduction Relation (1)



Let $s_1, s_2, \ldots$ denote guarded sums, i.e., non-deterministic processes of the form

$$\pi_1.p_1 + \cdots + \pi_n.p_n$$

Reduction Relation (1)



Let $s_1, s_2, \ldots$ denote guarded sums, i.e., non-deterministic processes of the form

$$\pi_1.p_1 + \cdots + \pi_n.p_n$$

Internal Step and Synchronization

$$\frac{}{\tau.p + s \longmapsto p}$$

$$\frac{}{(x(y).p_1 + s_1) \mid (\overline{x}\, z.p_2 + s_2) \longmapsto p_1[^z/_y] \mid p_2}$$

In both cases, the reduction step discards possible behaviors (in s, s_1 and s_2).

Reduction Relation (2)



Contextual Rules

$$\frac{p \longmapsto p'}{p \mid q \longmapsto p' \mid q}$$

$$\frac{p \longmapsto p'}{(x)p \longmapsto (x)p'}$$

The reductions of a process are preserved when placed in a larger context.

Reduction Relation (2)



Contextual Rules

$$\frac{p \longmapsto p'}{p \mid q \longmapsto p' \mid q} \qquad \frac{p \longmapsto p'}{(x)p \longmapsto (x)p'}$$

The reductions of a process are preserved when placed in a larger context.

Structural Congruence

$$\frac{p \equiv q \qquad q \longmapsto q' \qquad q' \equiv p'}{p \longmapsto p'}$$

Structural congruence can be applied before a reduction step (e.g. to reorder the elements of a sum) but also after (e.g. to remove an inactive process).

Example: Scope Extrusion with $\equiv$



We infer the reduction:

$$p \triangleq x(y).\overline{c}\, y \mid (b)\overline{x}\, b \longmapsto (b)\overline{c}\, b$$

Example: Scope Extrusion with $\equiv$



We infer the reduction:

$$p \triangleq x(y).\overline{c}\,y \mid (b)\overline{x}\,b \longmapsto (b)\overline{c}\,b$$

1. Since $b \notin \mathrm{fn}(x(y).\overline{c}\,y)$, we can enlarge the scope of b:

$$p \equiv (b)(x(y).\overline{c}\,y \mid \overline{x}\,b)$$

Example: Scope Extrusion with $\equiv$



We infer the reduction:

$$p \triangleq x(y).\overline{c}\,y \mid (b)\overline{x}\,b \longmapsto (b)\overline{c}\,b$$

1. Since $b \notin \mathrm{fn}(x(y).\overline{c}\,y)$, we can enlarge the scope of b:

$$p \equiv (b)(x(y).\overline{c}\,y \mid \overline{x}\,b)$$

2. We have $x(y).\overline{c}\,y \mid \overline{x}\,b \longmapsto \overline{c}\,b \mid \mathbf{nil}$, using the contextual rule we infer:

$$(b)(x(y).\overline{c}\,y \mid \overline{x}\,b.q) \longmapsto (b)(\overline{c}\,b \mid \mathbf{nil})$$

Example: Scope Extrusion with $\equiv$



We infer the reduction:

$$p \triangleq x(y).\overline{c}\,y \mid (b)\overline{x}\,b \longmapsto (b)\overline{c}\,b$$

1. Since $b \notin \mathrm{fn}(x(y).\overline{c}\,y)$, we can enlarge the scope of b:

$$p \equiv (b)(x(y).\overline{c}\,y \mid \overline{x}\,b)$$

2. We have $x(y).\overline{c}\,y \mid \overline{x}\,b \longmapsto \overline{c}\,b \mid \mathbf{nil}$, using the contextual rule we infer:

$$(b)(x(y).\overline{c}\,y \mid \overline{x}\,b.q) \longmapsto (b)(\overline{c}\,b \mid \mathbf{nil})$$

3. Since $b \notin \mathrm{fn}(\mathbf{nil})$, we can reduce the scope of b and remove $\mathbf{nil}$:

$$(b)(\overline{c}\,b \mid \mathbf{nil}) \equiv (b)\overline{c}\,b \mid \mathbf{nil} \equiv (b)\overline{c}\,b$$

Another Example: Non-Determinism

$$p \triangleq (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid x(u).\overline{u}\, v \mid \overline{x}\, z)$$

Observe: $\mathsf{fn}(p) = \{x, v, y\}$ and $\mathsf{bn}(p) = \{z, w, u\}$. Two possible reductions:

Another Example: Non-Determinism

$$p \triangleq (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid x(u).\overline{u}\, v \mid \overline{x}\, z)$$

Observe: $\mathsf{fn}(p) = \{x, v, y\}$ and $\mathsf{bn}(p) = \{z, w, u\}$. Two possible reductions:

1. Synchronization between the first and second components:

$$\begin{aligned} p &\longmapsto (z)(\mathrm{nil} \mid (\overline{u}\, v)[y/u] \mid \overline{x}\, z) \\ &= (z)(\mathrm{nil} \mid \overline{y}\, v \mid \overline{x}\, z) = p_1 \end{aligned}$$

2. Synchronization between the second and third components:

$$\begin{aligned} p &\longmapsto (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid (\overline{u}\, v)[z/u] \mid \mathrm{nil}) \\ &= (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid \overline{z}\, v \mid \mathrm{nil}) = p_2 \end{aligned}$$

Another Example: Non-Determinism

$$p \triangleq (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid x(u).\overline{u}\, v \mid \overline{x}\, z)$$

Observe: $\mathrm{fn}(p) = \{x, v, y\}$ and $\mathrm{bn}(p) = \{z, w, u\}$. Two possible reductions:

1. Synchronization between the first and second components:

$$\begin{aligned} p &\longmapsto (z)(\mathrm{nil} \mid (\overline{u}\, v)[y/u] \mid \overline{x}\, z) \\ &= (z)(\mathrm{nil} \mid \overline{y}\, v \mid \overline{x}\, z) = p_1 \end{aligned}$$

2. Synchronization between the second and third components:

$$\begin{aligned} p &\longmapsto (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid (\overline{u}\, v)[z/u] \mid \mathrm{nil}) \\ &= (z)((\overline{x}\, y + z(w).\overline{w}\, y) \mid \overline{z}\, v \mid \mathrm{nil}) = p_2 \end{aligned}$$

While $p_1 \not\longmapsto$, we do have $p_2 \longmapsto (z)(\overline{z}\, y \mid \mathrm{nil} \mid \mathrm{nil}) \equiv (z)\overline{z}\, y$.

Labeled Transitions

$$p \xrightarrow{\alpha} q$$

We have the following labels:

$\alpha ::= \tau$	internal action
$\mid x(y)$	input of a name y on x
$\mid \overline{x}\, y$	free output of name y on x
$\mid \overline{x}(y)$	bound output of name y on x

The definitions of $\mathsf{fn}(-)$ and $\mathsf{bn}(-)$ extend to actions. In particular:

- $\mathsf{fn}(x(y)) = \{x\}$
- $\mathsf{bn}(x(y)) = \mathsf{bn}(\overline{x}(y)) = \{y\}$

Action Prefix

$$\text{act}\quad \frac{}{\pi.p \xrightarrow{\pi} p}$$

In particular, the case of input requires a fresh name z:

$$x(y).p \xrightarrow{x(y)} p$$

$$x(y).p = x(z).(p[^z/_y]) \qquad \text{if } z \notin \text{fn}(p)$$

$$x(y).p \xrightarrow{x(z)} p[^z/_y] \qquad \text{if } z \notin \text{fn}(p)$$

Thus, z is just a placeholder, not an actual value.

Matching and Choice

Matching

$$\text{match}\ \frac{p \xrightarrow{\alpha} q}{[x = x]p \xrightarrow{\alpha} q}$$

- The rule checks the equality of names before enabling p.
  If the matching condition is not satisfied then the execution halts.

Matching and Choice

Matching

$$\text{match}\ \frac{p \xrightarrow{\alpha} q}{[x = x]p \xrightarrow{\alpha} q}$$

- The rule checks the equality of names before enabling p.
  If the matching condition is not satisfied then the execution halts.

Choice

$$\text{sumL}\ \frac{p \xrightarrow{\alpha} p'}{p + q \xrightarrow{\alpha} p'} \qquad \text{sumR}\ \frac{q \xrightarrow{\alpha} q'}{p + q \xrightarrow{\alpha} q'}$$

- Completely analogous to the rules for choice in CCS.
  Example: $\mathit{confirm}(y).([y = ok]p + [y = no]q)$

Parallel Composition

parL

$$\frac{p_1 \xrightarrow{\alpha} q_1}{p_1 \mid p_2 \xrightarrow{\alpha} q_1 \mid p_2} \ \mathsf{bn}(\alpha) \cap \mathsf{fn}(p_2) = \emptyset$$

parR

$$\frac{p_2 \xrightarrow{\alpha} q_2}{p_1 \mid p_2 \xrightarrow{\alpha} p_1 \mid q_2} \ \mathsf{bn}(\alpha) \cap \mathsf{fn}(p_1) = \emptyset$$

Example

Take the process $x(y).\overline{y}\,z \mid w(u)$.

Parallel Composition

parL

$$\frac{p_1 \xrightarrow{\alpha} q_1}{p_1 \mid p_2 \xrightarrow{\alpha} q_1 \mid p_2} \ \mathrm{bn}(\alpha) \cap \mathrm{fn}(p_2) = \emptyset$$

parR

$$\frac{p_2 \xrightarrow{\alpha} q_2}{p_1 \mid p_2 \xrightarrow{\alpha} p_1 \mid q_2} \ \mathrm{bn}(\alpha) \cap \mathrm{fn}(p_1) = \emptyset$$

Example

Take the process $x(y).\overline{y}\,z \mid w(u)$.

- The following transition is allowed:

$$x(y).\overline{y}\,z \mid w(u) \xrightarrow{x(v)} \overline{v}\,z \mid w(u) \qquad v \text{ is fresh } \checkmark$$

- But the following transition is not:

$$x(y).\overline{y}\,z \mid w(u) \xrightarrow{x(w)} \overline{w}\,z \mid w(u) \qquad w \text{ is not fresh! } \times$$

Communication

comL

$$\frac{p_1 \xrightarrow{\overline{x}\,z} q_1 \qquad p_2 \xrightarrow{x(y)} q_2}{p_1 \mid q_1 \xrightarrow{\tau} q_1 \mid q_2[z/y]}$$

comR

$$\frac{p_1 \xrightarrow{x(y)} q_1 \qquad p_2 \xrightarrow{\overline{x}\,z} q_2}{p_1 \mid q_1 \xrightarrow{\tau} q_1[z/y] \mid q_2}$$

Example

Consider the process $\overline{x}\,z.y(v) \mid x(y).\overline{y}\,z$. We have:

- $\overline{x}\,z.y(v) \xrightarrow{\overline{x}\,z} y(v)$
- $x(y).\overline{y}\,z \xrightarrow{x(y)} \overline{y}\,z$
- Therefore, by rule comL:

$$\overline{x}\,z.y(v) \mid x(y).\overline{y}\,z \xrightarrow{\tau} y(v) \mid \overline{z}\,z$$

Restriction



$$\text{res}\quad \frac{p \xrightarrow{\alpha} q}{(y)p \xrightarrow{\alpha} (y)q}\ y \notin \mathsf{fn}(\alpha) \cup \mathsf{bn}(\alpha)$$

- Process $(y)p$ can only perform actions that do not mention the name y.

Representing Scope Extrusion



The most interesting (and involved) part of the LTS.

Two ingredients:

- The label for bound output, denoted $\overline{x}(y)$, which declares that the restricted name y is being sent along x.
- Dedicated rules model the "opening" and "closing" of the scope of y.

Scope Extrusion (1)



open

$$\frac{p \xrightarrow{\overline{x}\, y} p'}{(y)p \xrightarrow{\overline{x}(w)} p'[w/y]}\ (y \neq x) \wedge w \notin \mathsf{fn}((y)p)$$

- By changing $\overline{x}\, y$ into $\overline{x}(w)$ (w fresh), the opened scope is "advertised".
- The scope is temporarily 'lost' after the transition to p'.

Scope Extrusion (1)



open

$$\frac{p \xrightarrow{\overline{x}\, y} p'}{(y)p \xrightarrow{\overline{x}(w)} p'[w/y]}\ (y \neq x) \wedge w \notin \mathsf{fn}((y)p)$$

- By changing $\overline{x}\, y$ into $\overline{x}(w)$ (w fresh), the opened scope is "advertised".
- The scope is temporarily 'lost' after the transition to p'.

closeL

$$\frac{p_1 \xrightarrow{\overline{x}(z)} q_1 \qquad p_2 \xrightarrow{x(z)} q_2}{p_1 \mid q_1 \xrightarrow{\tau} (z)(q_1 \mid q_2)}$$

closeR

$$\frac{p_1 \xrightarrow{x(z)} q_1 \qquad p_2 \xrightarrow{\overline{x}(z)} q_2}{p_1 \mid q_1 \xrightarrow{\tau} (z)(q_1 \mid q_2)}$$

- The scope is restored after communication.

Scope Extrusion (2)

open

$$\frac{p \xrightarrow{\overline{x}\, y} p'}{(y)p \xrightarrow{\overline{x}(w)} p'[w/y]}\ (\star)$$

closeL

$$\frac{p_1 \xrightarrow{\overline{x}(z)} q_1 \qquad p_2 \xrightarrow{x(z)} q_2}{p_1 \mid q_1 \xrightarrow{\tau} (z)(q_1 \mid q_2)}$$

Example

Consider the process $(y)\overline{x}\, y.\overline{y}\, a \mid x(v).v(b)$. We have:

- $(y)\overline{x}\, y.\overline{y}\, a \xrightarrow{\overline{x}(z)} \overline{z}\, a$ (via rule open)
- $x(v).v(b) \xrightarrow{x(z)} z(b)$ (via rule act)
- Therefore, by rule closeL:

$$(y)\overline{x}\, y.\overline{y}\, a \mid x(v).v(b) \xrightarrow{\tau} (z)(\overline{z}\, a \mid z(b))$$

Replication

Three different alternatives:

rep
$$\frac{p \mid !p \xrightarrow{\alpha} q}{!p \xrightarrow{\alpha} q}$$

rep
$$\frac{p \mid p \xrightarrow{\alpha} q}{!p \xrightarrow{\alpha} q \mid !p}$$

(Avoids infinite branching)

rep
$$\frac{p \xrightarrow{\alpha} q}{!p \xrightarrow{\alpha} q \mid !p}$$

(Input-guarded replication)

Example

$$!req(y).p \xrightarrow{req(y)} p \mid !req(y).p$$

Summary



Today:

- The π-calculus: extension of CCS with dynamic communication topologies.
- Key technical ingredients: name passing and scope extrusion.
- Two approaches to operational semantics:
  reduction semantics (closed systems) and LTS (open systems).

Next: Strong bisimilarity and a linear type system (Kobayashi et al.)



The End